中期答辩文稿（逐页详细版）

> 本文稿与 PPT 13 页正文 + 备份页逐页对应。
> 预计时长 8-10 分钟。每页标注了建议时长。
> 【括号中的内容】是动作提示或注意事项，不要念出来。

第 1 页：封面（15 秒）

各位老师好，我是计算机学院的武垚乐，我的毕业设计题目是《基于零知识证明的分布式推理可验证性研究》，指导教师是张宇超老师。下面我汇报一下中期进展情况。

第 2 页：课题背景与研究目标（1 分钟）

【翻到第 2 页，指向左侧“背景“部分】

首先说一下课题背景。

随着深度学习模型规模的持续增长，单节点推理在算力、时延和吞吐方面都面临明显瓶颈。分布式推理是提升推理效率的自然选择——把一个大模型切成多段，分给多台机器各算一部分。

但分布式推理引入了一个新的问题：**信任问题**。当推理任务分配到多个可能不完全可信的节点上时，我们怎么知道每个节点确实按照正确的模型做了计算？

零知识证明技术为这个问题提供了一种解决思路：它可以让计算节点生成一份密码学证明，证明“我确实按照规定的方式完成了计算“，而验证者不需要重新算一遍就可以验证这份证明的有效性。

【指向右侧“三项目标“】

根据任务书的要求，我的课题设定了三项核心目标：第一，构建一个分布式推理原型系统；第二，引入零知识证明来验证推理的正确性；第三，设计低开销优化策略来降低验证的计算成本。

总结来说，课题围绕“分布式推理、可验证性、低开销“这三条主线展开。

第 3 页：当前项目进展概况（45 秒）

【翻到第 3 页，指向三栏结构】

我先概要介绍一下目前的进展。

在“已完成“这一栏里：系统的 Master-Worker 分布式原型已经跑通了，支持 2 切片、4 切片和 8 切片三种配置；我实现了一个叫做 proof-bound output 的机制，它把零知识证明和推理输出做了数学绑定，这是系统最核心的安全设计，后面会详细讲。此外，我已经完成了 6 组实验并生成了图表。

“正在完善“的包括：统一实验管线和指标口径、收紧系统边界表述。

“下一步“计划是用主系统完整逻辑重跑关键实验、补充缺失的攻击场景、推进论文写作。

总体来说，原型已经完成，当前重点是实验收口和论文推进。

第 4 页：系统总体架构（1 分钟）

【翻到第 4 页，指向架构图】

接下来介绍系统是怎么设计的。

系统采用 Master-Worker 架构。Master 是中央调度节点，它负责三件事：统一调度推理请求、汇总各节点的结果、执行多层验证。

Worker 是工作节点，每个 Worker 持有模型的一个切片。Worker 有两种角色：一种是 **proof 节点**，调用 `/infer` 接口，执行完推理后还会生成一份完整的零知识证明——这份证明可以被 Master 独立验证；另一种是 **light 节点**，调用 `/infer_light` 接口，只做推理加哈希，速度快但安全保障弱一些。对于 light 节点，Master 可以随时发起随机挑战，要求它补交证明。

【指向校验路径部分】

验证方面，系统设计了三层校验加随机挑战的组合机制，我接下来详细讲。

总结：系统采用 Master-Worker 架构，将推理流和验证流分层解耦。

第 5 页：核心验证机制（1 分钟）

【翻到第 5 页，依次指向四个验证层】

这一页是系统最核心的设计。

第一层 L1，外部完整性检查：对每个节点的输入和输出做 SHA-256 哈希校验。这一层开销极低，可以快速发现非恶意的故障，但它不能对抗有意的篡改——因为恶意节点可以同时伪造输出和哈希。

第二层 L2，零知识证明验证：这是系统密码学安全的核心来源。proof 节点在 Halo2 算术电路中生成 EZKL 证明，Master 本地独立验证这份证明。

这里有一个关键设计叫 **proof-bound output**。简单来说，Worker 返回的推理输出不是自己独立算的，而是直接从证明的公开实例中提取的。这样一来，证明验证通过就意味着输出一定是正确的。恶意 Worker 无法在生成有效证明的同时返回篡改的输出。这把篡改从“事后检测“升级为“源头预防“。

第三层 L3，跨节点哈希链：前一个节点的输出哈希必须等于后一个节点的输入哈希，用于检测链路传输异常。

随机挑战：对 light 节点发起 `/re_prove`，要求它重新生成证明，提供概率性威慑。

总结：分层验证的思路是——proof 节点提供密码学保证，light 节点提供轻量完整性检查。

第 6 页：模块实现情况（30 秒）

【翻到第 6 页，快速扫过表格】

这一页快速过一下代码实现情况。

系统核心是 master.py 和 worker.py 两个文件，包含完整的三层校验、随机挑战和 proof-bound output。工具层是 utils.py，封装了 EZKL 的初始化、证明生成和验证。实验脚本有多个，覆盖了性能、安全性和隐私三个维度。另外建立了 18 项回归测试，全部通过。

需要说明的是，部分实验脚本使用的是简化评估管线，用于性能趋势分析，已在文档中做了标注。

代码模块体系完整，可以支撑后续的实验和论文写作。

第 7 页：主系统运行证据（1 分钟）

【翻到第 7 页，指向右侧的柱状图】

这一页展示系统的实际运行数据。

【指向图 p07_main_system_latency_breakdown.png】

右边这张图是 2 切片、4 切片和 8 切片配置下的端到端时延分解。蓝色部分是证明生成时间，橙色是验证时间，灰色是网络和 IO 开销。

可以看到三个关键观察：

第一，2、4、8 切片三种配置都成功跑通了，均可正常产生最终输出。

第二，证明生成时间占端到端时延的约 85%，是绝对的主要瓶颈。验证时间很短，大约只占 7%。这说明“降低证明开销“确实是最值得优化的方向。

第三，在故障注入场景下——也就是我们在最后一个切片上人为注入了篡改——篡改的输出被 proof-bound 机制在源头替换成了正确值。系统不是“检测到了错误然后报警“，而是“篡改根本就没能传播出去“。

【如果页面有小图 p07_side_proof_bound_prevention.png，可以指一下】

右下角这张小图更直观地展示了正常运行和故障注入两种场景的对比。可以看到，故障注入下的时延甚至略低，因为被篡改的输出直接被 proof 绑定的正确值替代了。

总结：系统在多种配置下均可稳定运行，proof-bound 机制有效预防了响应层篡改。

第 8 页：选择性验证结果（1 分 30 秒）

【翻到第 8 页，指向主图，这是重点页】

这一页是我认为研究价值最高的部分——选择性验证。

为什么需要选择性验证？ 因为前面我们看到了，证明生成占了 85% 的时延。如果每个节点都生成证明，8 切片下端到端需要 15.5 秒。能不能不让所有节点都生成证明，只选一部分？

Edge-cover 策略：我们的做法是保证一个约束——对于流水线上任意相邻的两个节点，至少有一个是 proof 节点。首尾两个节点必须是 proof 节点。这样即使某些节点是 light 节点，与它相邻的 proof 节点也可以通过哈希链和证明验证来约束它。

【指向图表】

这里需要澄清一个重要概念：**verify_ratio 和 actual_proof_fraction 不是同一回事**。verify_ratio 是用户设定的期望验证比例，比如设成 25%。但经过 edge-cover 策略调整后，实际的 proof 覆盖率可能是 62.5%——因为策略需要满足边覆盖约束。

看具体数据：在 8 切片配置下，verify_ratio 从 100% 降到 25%，端到端时延从 15.5 秒降到 9.1 秒，降幅约 41.5%。4 切片下也有类似的趋势，降幅约 37%。

右下角这张补充图直接展示了开销降低的百分比：36% 到 42% 之间。

总结：选择性验证在保持边覆盖约束的前提下，端到端开销降低了约 36% 到 42%。

第 9 页：攻击检测结果（1 分钟）

【翻到第 9 页，指向攻击处理图】

这一页展示安全性验证的结果。

先说攻击模型：我们当前验证的是**响应层篡改**——就是 Worker 确实按照正确的模型做了推理，但在返回结果时故意篡改了输出。我们设计了四种篡改方式：tamper 是在输出上加 999，skip 是返回全零，random 是返回随机数，replay 是返回一个固定值。

【指向图中的柱状图】

可以看到，在两种验证预算（100% 和 50%）下，四种攻击全部被成功处理。柱子上标的 “P” 代表 prevention——预防，不是检测。因为恶意节点恰好落在 proof 节点位置，篡改的输出被 proof-bound 机制在源头替换成了正确值。

【语气切换为严谨】

但我必须指出重要的限定条件：

第一，这个结论只在当前的攻击模型下成立——也就是“响应层篡改加独立恶意节点“。

第二，在我们的实验设置中，故障节点恰好落在了 proof 节点的位置，所以全部被预防。如

果故障节点落在 light 节点位置，就需要依赖 L1 检测和随机挑战，目前我们还缺少这部分的实验数据。

第三，这个结论不应该被泛化为“系统可以检测所有恶意行为“。

第 10 页：可见性模式对比（45 秒）

【翻到第 10 页，指向左右两张图】

这一页对比了三种可见性模式的开销差异。

all_public 是基准模式，输入数据以明文出现在证明中。**hashed** 模式用 Poseidon 哈希保护输入数据。**private** 模式下输入完全不出现在证明的公开实例中。

【指向左图 p10_main_visibility_time.png】

左边这张图是证明时间和验证时间的对比。可以看到 hashed 模式的证明时间明显更高，约为 all_public 的 1.6 倍。这是因为 Poseidon 哈希需要在零知识电路中额外引入约束。而 private 模式和 all_public 基本持平，这是因为 private 只是简单地不公开输入，不需要额外的电路计算。

【指向右图 p10_side_visibility_size.png】

右边是产物大小的对比。hashed 模式的 proof 大小增加了约 18%。

总结：三种模式的开销差异已经可以定量比较，这为后续的隐私-性能权衡研究提供了数据支撑。

第 11 页：问题与系统边界（45 秒）

【翻到第 11 页，语气变得诚恳】

这一页我主动列出目前存在的问题和系统边界。

左边是待解决的问题：第一，部分实验用的是简化管线，还没有和主系统完全统一；第二，verify_ratio 和 actual_proof_fraction 需要在所有结论中明确区分；第三，缺少 light 节点被攻击的实验数据；第四，模型规模比较小，是 480 参数的全连接网络。

右边是系统边界：第一，Master 是可信假设，被攻破则全线失效——这是中心化设计的固有限制；第二，light 节点不具备强密码学约束；第三，跨节点的中间数据在传输时是明文的；第四，当前是单机多进程模拟，不是真正的多机部署；第五，攻击模型只覆盖了响应层篡改。

我认为主动暴露这些边界是研究诚实的体现。原型已经可以运行，但确实需要继续收紧实验口径和安全边界。

第 12 页：下一步计划（30 秒）

【翻到第 12 页】

近期 1-2 周内的工作：用 Master 的完整逻辑重跑关键实验，补充 light 节点被攻击的场景，统一指标口径。

2-4 周的工作：收紧论文措辞，完善性能和安全性分析，视情况补充更大模型的实验。

最终目标是形成完整的实验矩阵，完成毕业论文和终期答辩材料。

第 13 页：总结（15 秒）

【翻到最后一页】

总结一下。我目前已经完成了分布式推理原型系统，接入了 EZKL 零知识证明，实现了 proof-bound output 机制，完成了 6 组阶段性实验。

当前已经形成了一个可运行、可验证、可测量的研究原型。后续重点是统一实验口径、收紧边界表述、推进论文写作。

以上就是我的中期汇报，谢谢各位老师。

预判问答准备

以下是老师可能追问的问题及建议回答。

Q1：你说的 proof-bound output 到底解决了什么问题？能具体说一下吗？

原来的设计是这样的：Worker 自己用 ONNXRuntime 算一个结果，同时用 EZKL 生成一份证明。但这两个过程是独立的——Worker 返回的输出是 ONNXRuntime 算的，证明是 EZKL 生成的。一个恶意的 Worker 完全可以让 EZKL 生成正确的证明，但返回一个被篡改的输出。Master 验证了证明，发现证明是对的，就信了这个输出——但实际上输出是假的。

proof-bound output 的解决方案是：Worker 不再用 ONNXRuntime 的输出，而是直接从证明的公开实例中提取输出。证明里面包含了正确的输出值（rescaled_outputs），Worker 直接用这个作为返回值。Master 也独立从证明中提取输出做交叉验证。这样就把证明和输出绑死了——证明对，输出就一定对。

Q2：edge-cover 策略具体是怎么选节点的？

策略的核心约束是：对于流水线上任意相邻的两对节点 (i, i+1)，至少一端必须是 proof 节点。同时，首尾必须是 proof 节点。

具体实现是：先把首尾选为 proof，然后从前往后扫描，如果连续出现两个 light 节点，就把后面那个改成 proof。这样保证了最大 light gap 为 1。

Q3：你的实验中攻击节点都在 proof 位置，那 light 节点被攻击怎么办？

这确实是目前实验的一个不足。如果攻击节点落在 light 节点位置，它不会生成证明，所以 proof-bound 机制就不适用了。这时候依赖的是：L1 的哈希校验（但恶意节点可以绕过）、L3 的哈希链检查（可以发现传输异常）、以及随机挑战（Master 可能抽查这个 light 节点，要求它补交证明）。这些加在一起提供的是概率性保障，不是确定性保障。这也是我在下一步计划中要补充的实验。

Q4：为什么 hashed 模式比 private 模式还慢？

因为 hashed 模式需要在零知识电路内部计算 Poseidon 哈希。Poseidon 是专门为 ZK 电路设计的哈希函数，但它毕竟会增加电路的约束数量（约增加了 1.6 倍的证明时间）。而 private 模式只是简单地把输入标记为 private（不出现在公开实例中），这不需要额外的电路计算，所以开销和 all_public 基本一样。

Q5：480 参数的模型也太小了吧？实际意义在哪里？

这个确实是一个局限。480 参数的全连接网络是用来验证系统概念的——它能跑通完整的切片、推理、证明、验证流程，可以测量各种开销和安全性指标。但它不能直接证明系统在大模型上也能工作。

不过需要指出的是，EZKL 本身已经被业界用于更大规模模型的证明。我们的系统只是在 EZKL 之上做了分布式包装。如果 EZKL 能处理更大的模型切片，我们的系统理论上也可以。后续我计划补充 MNIST MLP 的实验。

Q6：切片本身会不会引入误差？

【备份页 A】

不会。我专门做了 P4 实验来验证这一点：把 PyTorch 的切片串联输出和完整模型输出做逐元素对比，所有误差指标（L1 距离、L2 距离、最大绝对误差、相对误差）全部为零，是 bit-exact 的。

但需要注意，这只验证了 PyTorch 层级的切片一致性。经过 ONNX 导出和 EZKL 量化之后可能会引入量化误差，这一部分目前还没有做专门的实验。

Q7：你说 Master 是可信的，这不是一个很强的假设吗？

是的，这确实是最强的假设之一。Master 执行所有的验证逻辑——如果 Master 被攻破，攻击者可以直接绕过所有验证。

这是中心化验证架构的固有限制。在去中心化场景中，可以考虑去中心化验证（比如链上验证），但这超出了当前课题的范围。目前的工作重点是在 Master 可信的前提下，验证分布式推理 + ZKP 这条技术路线的可行性。